

AMC .amc Execution Specification

Language-agnostic front-end spec: .amc source → per-path payoff callables over SimulatedMarketData

June 2026

AMC .amc Execution Specification

A language-agnostic specification for compiling an .amc product into per-path payoff callables over a SimulatedMarketData contract.

This document specifies the **front end**: how .amc source text becomes an executable **Product** — an ordered list of events whose payoff/decision amounts are *callables* that, given a per-date market snapshot and a per-path state object, return length-*N* per-path vectors. It is the missing third pillar between the **DSL Payoff Reference** (the surface language and the function catalogue) and the **AMC Engine Specification** (the Longstaff-Schwartz pricing and exposure core, which *consumes* the **Product** this document produces).

Nothing here is tied to a particular programming language. All values are **per-path vectors** of a fixed length *N* (the path count); the only required runtime types are a real-vector type **Vec** (length *N*), scalars, and the **SimulatedMarketData** query object defined in §2. Where the reference implementation uses a specific numeric idiom, the idiom is stated abstractly (e.g. “element-wise maximum”, “select-by-mask”) rather than as a library call.

The four layers are independent and can be implemented in order:

1. **Lexing & parsing** (§3) — source text → a parse tree.
2. **Lowering / resolution** (§4) — parse tree → a *resolved product*: a flat, ordered event list whose amounts are **fully-substituted expression strings** (schedules expanded, macros substituted, functions inlined).
3. **Evaluation** (§5–§6) — an expression evaluator over a per-date snapshot and per-path state, plus the **name→market dispatch** (§6) that binds every name and built-in call to a **SimulatedMarketData** query.
4. **Execution model** (§7) — how the resolved events are run: the (date, seq) ordering, the forward state pre-pass, and the payoff-callable contract the engine consumes.

A correct implementation of all four reproduces the reference engine’s per-path amounts bit-for-bit given the same **SimulatedMarketData**.

1. Object model

1.1 The resolved product

Lowering (§4) produces a **resolved product** with these parts:

- **name** — an identifier.
- **currency** — the settlement currency (mandatory; drives discounting in the engine, not here).

- `underlyings` — the ordered list of tradable factor names the payoffs read.
- `state` — a list of `(name, init_expr, is_regressor)` declarations; `init_expr` is an expression evaluated **once** at `t0` to seed the per-path state vector.
- `events` — a **flat, totally-ordered** list of events (§1.2). All structural sugar (`for`, `schedule`, `schedule_div`, instrument blocks, `func`) has been expanded away; every remaining amount/condition is a plain expression string in the sub-language of §5.

The engine never sees the source text — only this resolved product.

1.2 Event kinds

Every event carries a `date` (a year-fraction from `as_of`) and an integer `seq` (§7.1). There are three kinds:

Kind	Carries	Effect
Fixing	a <code>do</code> string: one or more <code>name = expr</code> assignments, <code>;</code> -separated	mutates per-path state; emits no cashflow
Payment	an amount expression, a boolean <code>discounted</code> flag, optional cross-currency (<code>currency</code> , <code>fx</code>) tag	emits a per-path cashflow vector
Decision	<code>immediate</code> (intrinsic) <code>expr</code> , <code>alive</code> <code>expr</code> , <code>forced_stop</code> <code>expr</code> , a <code>polarity</code> (<code>max</code> = holder / <code>min</code> = issuer), an optional <code>regressors</code> list, physical-settlement flags	an optimal-stopping decision resolved by the engine's regression

A `Fixing`'s `do` may contain several assignments; they execute left-to-right (§7.3). A `Payment` with `discounted = false` is paid undiscounted (the `*_undisc` surface forms). The `Decision` fields are expressions like any other and are evaluated by the same evaluator; the engine supplies the continuation value.

1.3 Per-path state

State is a mutable map `label → Vec`. It holds: (a) declared state variables, seeded at `t0`; (b) **auto-created** accumulators introduced by string-labelled built-ins (`RUNNING_*`, `SUP/INF`, `SNAPSHOT`, `BARRIER_SURVIVAL`, ...) on first use; (c) array-state vectors (`name[K]`). Every state label, declared or auto-created, is visible to later expressions as a bare name (§6.1). State persists along each path for the whole simulation and is built in a single forward pass (§7.2) **before** the engine's backward pricing sweep.

2. The `SimulatedMarketData` contract

The evaluator is parameterised over a **market object** that answers per-path queries. This is the entire coupling between the payoff language and the simulation: implement payoffs against *this* interface and they are independent of how paths were produced. Two views are used:

- **snap** — a per-date *snapshot*: `snap[name]` returns the `Vec` of factor `name` sampled at the snapshot's date. This is the fast path for a bare underlying read at the current event date.

- **md** — the full market object, used when a built-in needs an off-date or derived quantity. Every method returns a `Vec` of length `N`. Time arguments are year-fractions; where a method takes both `t` and `T`, `t` is the observation time and `T` the maturity.

2.1 Required query surface

Method	Returns (per path)
<code>value(name, t)</code>	value of factor <code>name</code> at <code>t</code> , in its native currency (the generic accessor behind a bare underlying)
<code>spot(asset, t=0)</code>	spot of an equity/FX asset at <code>t</code> (at <code>t=0</code> , a constant vector)
<code>in_currency(name, t, ccy)</code>	<code>value(name, t)</code> converted into <code>ccy</code> using the path FX
<code>forward(asset, t, T)</code>	forward $F(t, T)$ of an equity/FX asset observed at <code>t</code>
<code>bond_price(ccy, t, T)</code>	zero-coupon bond $P(t, T)$ in <code>ccy</code> (the single “rate door”)
<code>discount_factor(ccy, t, T)</code>	synonym of <code>bond_price</code> at the resolver layer
<code>forward_rate(ccy, t, t1, t2)</code>	simply-compounded forward rate seen at <code>t</code> for <code>[t1, t2]</code> (drives LIBOR/CMS/OIS)
<code>fx_spot(pair, t=0), fx_forward(pair, t, T)</code>	FX spot / forward for <code>pair</code> (e.g. “EURUSD”)
<code>cash(ccy, t)</code>	money-market account $B(t) = \exp(\int_0^t r \, ds)$ under the pricing measure
<code>inv_cash(ccy, t)</code>	deflator $1/B(t)$ (preferred where a stochastic discount is needed)
<code>currency_of(name)</code>	the native currency code of a factor (scalar metadata)
<code>alive(issuer, t), survival(issuer, t)</code>	per-path 0/1 alive-indicator / survival value of a credit issuer
<code>forward_survival(issuer, t, T)</code>	forward survival probability $t \rightarrow T$
<code>recovery_rate(issuer), default_time(issuer)</code>	recovery and default-time of an issuer

2.2 Resampling semantics of `value`

`value(name, t)` is **not** a raw grid lookup; it resolves `t` against each factor’s *resample rule*, which the implementation must honour for off-grid `t`:

- **interp / log_interp** — linear (resp. log-linear) interpolation between the bracketing grid columns. Used for Markov factors (equity, FX, commodity, rate-factor levels).
- **ffill** — hold-last: the value at the latest grid date $\leq t$. Used for path-dependent state (running extrema, survival, accumulators).
- **recompute** — replay the product’s update at `t` if a recompute rule is registered, else fall back to `ffill`.
- **none** — exact grid column only; no off-grid resampling.
- A name with no registered rule defaults to **interpolation**.

A bare underlying read at the current event date (the common case) takes the snapshot fast path `snap[name]` and needs no resampling.

2.3 The pricing-measure numéraire

`cash/inv_cash` define the discount used by the engine, not by the payoff evaluator. For deterministic rates, `bond_price/cash` reduce to the initial curve $P(0, \cdot)$ and $\exp(\int_0^t r)$ on that curve; for stochastic rates they read the simulated short-rate factor and build B by trapezoidal integration over the shared date grid. The evaluator only *reads* these via the rate built-ins of §6; the engine builds $B(t)$ itself for discounting.

3. Lexing and parsing

The surface grammar is fully specified in the **DSL Payoff Reference, Part 6** (EBNF, lexical conventions, date/tenor/frequency tables). This document does not restate it; it specifies only what the *execution* layers add. The minimum the parser must produce for the lowering layer:

- **Lexical rules.** `#` line comments and `/* ... */` block comments; `//` is floor-division, **not** a comment. Statements separated by newlines or `;`. Declared identifiers (underlyings, state, `let`, groups, labels) are case-**sensitive**; built-in **function names are case-insensitive** and are canonicalised to upper case before lowering. Numbers may carry a `%` suffix (`3% = 0.03`) in settings. Reserved value names: `True`, `False`, `None`, `pi`, `e`, and the time origin `t0` (`= as_of`, year-fraction 0).
- **Date atoms.** A date is a float year-fraction, a tenor matching `[+-]?d+(\.\d+)?` [DWMYQSA] (chainable: `1Y6M`, `1Y+6M`), an ISO date string, `t0/as_of`, or a bare identifier (a `let` constant or a `for`-loop variable). Tenor units as year-fractions: `D=1/365.25`, `W=7/365.25`, `M=1/12`, `Y=1`; `Q=0.25`, `S=0.5`, `A=1` as period codes. **Arithmetic on a name is not allowed in a date slot** (at `t-3M` is illegal); relative offsets go inside the payoff (at `t { ... f(t-0.25) ... }`).
- **Expression grammar.** The payoff sub-language (operators, calls, comprehensions, ternary) is the EBNF of Payoff-Reference §6.3. A call argument is `arg ::= [ NAME "=" ] expr`, so positional and `name=value` **keyword** arguments share one production; the recognised argument names per built-in are the signature catalog in the Function Coverage Checklist. Product-level declarations include the credit blocks `tranche_pricer NAME { ... }` and `CDO NAME { ... }` and the `pricing = closedform | analytic` setting (Payoff-Reference Part 6 / Part 7). The parser emits an expression tree per amount/condition; the evaluator of §5 walks it.

The output of this layer is a parse tree whose event bodies still contain structural sugar; the lowering layer removes it.

4. Lowering (resolution)

Lowering rewrites the parse tree into the **resolved product** of §1.1. It is a sequence of deterministic transformations; an implementation must reproduce each exactly, because the engine's numbers depend on the *expanded* form, not the surface form. The transformations, in the order the reference applies them:

4.1 Constant and parameter folding

`let` bindings fold to literal values and are substituted wherever the name appears in an expression or a date slot. `groups name = [A, B, ...]` records a named basket used by rainbow aggregators and comprehensions.

Static vectors. A `let NAME[K] = [...]` binding (optional shape `[K]` or `[R][C]`; nested allowed) folds its elements to a compile-time list of values. The shape, when declared, is asserted against the literal (length mismatch or a non-rectangular nested literal is a compile error). Two lowering rewrites then apply wherever the vector name appears:

- a subscript `v[i]` (`v[i][j]`) whose index folds to an integer $\rightarrow$ the literal element (out-of-range index is a compile error);
- a bare `v` used as a list argument $\rightarrow$ the literal list `[v0, ..., v_{K-1}]`.

After these rewrites no vector name remains; the result is identical to writing the literals inline (zero runtime cost, no state-map entry).

SIZE family. `SIZE(x)`/`LEN(x)`/`LENGTH(x)` fold to the compile-time integer length of `x`; `NROWS(x)`/`NCOLS(x)` to the row / first-row length of a nested vector. `x` may be a `let`/mutable vector, a literal list, a `groups name`, a `let`-bound `schedule(...)` (its generated date count), or a `state` array (its declared slot count `K`). The fold makes `SIZE` usable as a range bound, so `range(0, SIZE(v))` is length-safe. **Boundary:** `SIZE(...)` as a *call* on a `schedule_div(...)` or any runtime per-path value is a compile error — those lengths are market-deferred / per-path and not known at lowering. (Distinct from the *bare* token `SIZE` inside a `schedule_div` body, which is the engine- substituted dividend count; see §4.5.)

Compile-time mutables. A mutable `NAME = <const>` (scalar) or mutable `NAME[K] = [...]` (vector) seeds a value in a *mutable symbol table* held alongside the immutable `let` environment. As the resolver walks events in `(date, seq)` order, each `set <mutable> = <const-expr>` (or `set v[i] = ...`) updates that table; every expression referencing the mutable substitutes its current value (a scalar $\rightarrow$ its literal; a vector element via the subscript rewrite above). Two invariants make this a *compile-time* construct — a single value, not per-path:

1. the assignment RHS (and any slot index) must fold to a compile-time constant — it may reference `let`, other mutables, `SIZE`, and arithmetic, but **not** an underlying, `state`, the event time `t`, or any per-path value (violation is a compile error); and
2. a mutable may be assigned only where its loop is **unrolled at lowering** — at top level, inside a compile-time `for v in [<list>]/range` loop, or inside a `for v in schedule(...)` whose dates are computable at lowering. The latter is **every plain or calendar/bda-rolled schedule** (`start/end/freq, n=`, or `dates=[...]`): the calendar roll is static (holiday calendar + `as_of`), so the resolver enumerates the dates via the engine's own date generator (`_generate_schedule_dates`) — guaranteeing the unrolled dates are identical to the template path. The schedule is then unrolled per date exactly like a list loop. The **only** disallowed case is `schedule_div(...)` (dividend-driven dates, market-deferred $\rightarrow$ nothing to unroll); a mutable there, or under a runtime `if/mask`, is a compile error. A schedule body that sets no mutable keeps the compact single-template lowering (resolved size independent of date count).

After lowering no mutable remains (like `func` inlining, it vanishes); the resolved product contains only the substituted literals. A per-path counter that must advance on a data-dependent condition is **not** a mutable — it is `state + RUNNING_COUNT/RUNNING_SUM`.

4.2 Function inlining (`func`)

A user `func NAME(params) { body }` is **pure** and **inlined** at every call site, producing a single expression: α -rename locals, substitute arguments, unroll compile-time `for` folds, and lower `if/else` to nested ternaries (`a if c else b`). Recursion is rejected; built-in names may not be shadowed. After inlining no `func` remains. (Full semantics: Payoff-Reference “User-defined functions”.)

4.3 Comprehensions and `range`

A comprehension `[expr for v in iterable [if cond]]` is **expanded at compile time** into an explicit list when the iterable is a literal list, a declared `groups` name, or a `range(...)` of integer-resolvable bounds (a bound may be a `let` that folds to an integer). The expansion substitutes each element for `v`. (Run-time comprehensions over `Vec` data are not produced; everything is unrolled.)

4.4 Schedule expansion (`schedule / for ... in schedule`)

`schedule(start, end | n, frequency, ...)` generates a set of dates and emits the block body **once per date**:

- **Date generation.** From `start` stepping by the frequency’s year-fraction ($A=1, S=0.5, Q=0.25, M=1/12$) up to `end` (inclusive within tolerance), or `n` periods from `start`. `day_count`, `calendar`, `bda`, `stub` pass through to the engine verbatim and affect only accrual/rolling, not the count.
- **Loop variable.** In the iterator form `for t in schedule(...)`, the loop variable binds to each generated **date** (a scalar year-fraction), available inside the body as a bare name.
- **Ordering.** Each generated event is placed at its date; multiple statements in the body keep declared order via `seq` (§7.1).

4.5 Dividend-schedule expansion (`schedule_div / for ... in schedule_div`)

`schedule_div(underlying, start, end, drive_by, ...)` iterates the **discrete dividend schedule** of `underlying` (resolved from the market’s dividend spec), emitting the body once per dividend in the ex-date window `[start, end]`. Per dividend `i` with ex-date `t_exi`, cash `Ci`, yield `Yi`, pay-date `t_payi`:

1. **Cum-div snapshot.** A hidden Fixing is injected at `t_exi -  $\epsilon$` ($\epsilon = 10^{-9}$) that snapshots the underlying into a hidden state label — the **cum-div** (pre-drop) spot. `DIV_AMOUNT` reads this label.
2. **Macro substitution.** In the body templates, the dividend macros are substituted (§4.6).
3. **Flow date.** The body event fires at the **driving date**: `t_exi` if `drive_by = ex_date`, else `t_payi`.
4. **Iterator binding.** In `for t in schedule_div(...)`, the loop variable binds to the driving date **and the body event fires there**: for `drive_by = ex_date` it binds to `t_exi -  $\epsilon$` (the cum-div instant), so a bare underlying read in the body is the **pre-drop** spot — making `DIV_C + DIV_Y · S` reconstruct `DIV_AMOUNT` exactly; for `pay_date` it binds to `t_payi`.
5. **Index tokens.** Two whole-identifier tokens are substituted per dividend at expansion time: `current` → the 0-based dividend index `i`, and `SIZE` → the dividend count `n`. They survive the front-end resolver as bare names (the resolver admits `current/SIZE` only

inside a `schedule_div` body) and are replaced when the dividend dates become known. This is the `schedule_div` analogue of a per-iteration counter — a compile-time mutable is impossible here because the dividend dates are market-deferred, so `current` is the counter and `SIZE` the count. They compose in arithmetic and as vector indices (`coupons[current]`, `current/(SIZE-1)`).

The window always filters by **ex-date**; `drive_by` only moves where flows fire.

4.6 Dividend-macro substitution

Inside a `schedule_div` body, per the bound dividend's (C_i , Y_i , t_{ex_i} , t_{pay_i}):

Macro (surface)	Lowers to
<code>DIV_AMOUNT(eq)</code>	$(C_i + Y_i \cdot Scum)$ where <code>Scum</code> is the hidden cum-div snapshot label
<code>FWD_DIV_AMOUNT(eq)</code>	a compile-time scalar $C_i + Y_i \cdot F(obs_time, t_{ex_i})$ from the analytic forward
<code>DIV_C</code> or <code>DIV_C(eq)</code>	the scalar C_i (call form rewrites to the bare token)
<code>DIV_Y</code> or <code>DIV_Y(eq)</code>	the scalar Y_i
<code>DIV_T_EX</code> / <code>DIV_T_PAY</code>	the scalars t_{ex_i} / t_{pay_i}
loop variable (iterator form)	the bound driving-date value (§4.5)

Outside any `schedule_div`, two aggregate macros are pre-substituted over the ex-date window: `SUM_FWD_DIVS(eq, a, b)` → a compile-time scalar; and `SUM_REALIZED_DIVS(eq, a, b)` → an auto-injected hidden `schedule_div` + running-sum accumulator, lowered to that accumulator's state label.

4.7 Fixed-inception ratio anchors (`RATIO/PERF`, `reset=false`)

`RATIO(S, reset=false, fixing=F)` (and `PERF`) is rewritten to use a hidden **anchor** state label captured once at the fixing time F (t_0 by default): `RATIO` → S / anchor ; `PERF` relative → $S / \text{anchor} - K$; `PERF` absolute → $S - K \cdot \text{anchor}$. A hidden Fixing that snapshots the anchor at F is injected. The rewrite distributes over `groups`. Plain `reset=true` (the default) is left untouched.

4.8 Instrument blocks

Declared instruments (`swap`, `bond`, `forward`, `convertible`, ...) self-expand into their constituent Payment/Decision events (coupon legs, redemption, exercise windows). The expansions are product-specific and specified in the Payoff Reference; the result is ordinary events in the resolved list.

4.9 Cross-currency tagging

A `receive/pay` with a trailing `currency = C [fx = PAIR]` tags the Payment with that currency and FX pair; at evaluation the amount is converted into the product currency via the FX path (§6).

4.10 Sequencing within an event

Statements inside one `at { }` block keep **declared order**. A `set` written **after** a `receive` in the same block is scheduled so the payment sees the *pre-update* state (§7.3). The lowering records this as ordering metadata (`seq`), not by nudging dates.

The output is the resolved product: a flat event list whose amounts are expression strings in the \$5 sub-language, with all names resolvable to an underlying, a (declared or hidden) state label, a `let/loop` scalar, or a built-in.

5. The expression evaluator

The evaluator turns one resolved expression string into a `Vec` (or a scalar that broadcasts to a `Vec`), given an **environment** `env` (§6.1) that maps names to values and built-ins to callables. It is a recursive walk over the expression tree. Every operation is **element-wise over paths**; a scalar operand broadcasts against a `Vec`.

5.1 Node semantics

Node	Evaluation
Literal	the constant value (number, string, <code>True/False/None</code>)
Name	<code>env[name]</code> if present; the constants <code>True/False/None</code> ; otherwise an error (an unresolved name is a compile-time bug in lowering)
Binary op <code>+ - * / // % **</code>	element-wise; <code>**</code> is right-associative; integer/float follow the host's numeric promotion. <code>/</code> is true division
Unary <code>+ -</code>	element-wise negation / identity
Comparison <code>< <= > >= == !=</code>	element-wise; returns a 0/1 mask <code>Vec</code> . Chained comparisons (<code>a < b < c</code>) fold to the conjunction of adjacent pairs
Boolean <code>and or</code> (keywords)	scalar logical ops only — defined on scalars, not on <code>Vec</code> ; using them on per-path masks is a usage error (they do not act element-wise)
Bitwise <code>& \ ^ ~</code>	element-wise mask AND / OR / XOR / NOT — the correct way to combine per-path conditions
Ternary <code>a if c else b</code>	element-wise select: <code>select(c, a, b)</code> (both branches evaluated, then masked)
List <code>[...]</code>	a host list of evaluated elements (used as a call argument, e.g. for aggregators/groups)
Subscript <code>x[i] / x[i:j]</code>	if <code>x</code> is a 2-D array-state (<code>N × K</code>), <code>x[i]</code> selects column <code>i</code> as a <code>Vec</code> ; otherwise an ordinary index/slice of a list or vector
Call <code>f(a, ...)</code>	look up <code>f</code> in <code>env</code> (must be a registered built-in callable); evaluate arguments; apply (§6). Indirect calls are forbidden — the callee must be a bare name. Keyword arguments are allowed only for the whitelisted kwarg-bearing families (e.g. <code>SWAP</code> , the <code>*_EXCLUDING</code> family, the <code>init=/cap=/floor=</code> accumulators); all other calls are positional-only

5.2 Determinism rules

- **Both ternary branches are evaluated** and combined by mask. There is no short-circuiting. This is required: a payoff must be defined on every path.
- **and/or never act on vectors.** Lowering and authors use `& | ~` for per-path logic; the evaluator must not silently element-wise the keyword forms.

- **Comparisons yield 0/1**, not booleans, so they compose arithmetically (`mask * cashflow`) and feed `where/ternary` identically.
- **Vectorisation/broadcasting** must match the host reference exactly at the boundaries that matter numerically (e.g. integer powers, division). Where the reference standardises or reduces, follow the same convention (population vs sample, tie-breaks) — see the Engine Spec’s cross-language appendix.

6. Name and call dispatch — the market binding

This is the contract that ties the language to `SimulatedMarketData`. The evaluator’s `env` is rebuilt **per event** (at that event’s date), by `make_env(snap, state, t)`:

6.1 Building `env`

In this order (last write wins; collisions are rejected at lowering):

1. **Underlyings.** For each declared underlying `name`, bind `env[name] = snap[name]` — the per-path value at the **current event date**. (A proxy underlying defined as `scale · leaf + shift` binds to `scale · snap[leaf] + shift`.) This is the rule “*a bare underlying name is its simulated value at the current event’s date.*” There is no `SPOT(asset, t)` surface function — to read a level elsewhere, snapshot it (§6.4) or place the event at that date.
2. **State variables.** For each declared state `label`, bind `env[label] = state[label]`.
3. **Auto-created state.** For every label present in the state map (accumulators from `RUNNING_*`, `SUP/INF`, `SNAPSHOT`, `BARRIER_SURVIVAL`, ...) not already bound, bind it — so later expressions read accumulators transparently.
4. **Baskets / groups** as needed (price baskets follow absolute-basket conversion semantics, converting each component into the product currency).
5. **The event time `t`.** Bind `env["t"] / t0` last so it cannot be shadowed.
6. **Built-in functions.** Bind every built-in name (§6.2–§6.5) to a callable closed over `(snap, md, state, t, product_ccy, ...)`.

A bare underlying is therefore a value, not a call; a built-in is a call. The evaluator never queries the market directly — it only reads `env`; the market coupling lives entirely in the closures bound in step 6.

6.2 Spot / value / forward / FX built-ins → market methods

Each built-in resolves to a `SimulatedMarketData` query (§2). The table gives the **surface call** and the **method it dispatches to**:

Surface call	Dispatches to	Notes
bare <code>NAME</code> (underlying)	<code>snap[NAME]</code> ($\equiv$ <code>value(NAME, t_event)</code>)	current-date level, native ccy
<code>FORWARD(asset, T)</code>	<code>forward(asset, t_event, T)</code>	per-path forward to <code>T</code>
<code>FIX(name) / FIXING(name, when) / PAST_FIX(name, when)</code>	observed fixing: current <code>snap[name]</code> , or a past observed fixing at <code>when</code>	historical fixings from the market’s fixing store
<code>FX(pair) / FX_SPOT(pair)</code>	<code>fx_spot(pair, t_event)</code>	per-path FX level
<code>FX_FWD(pair, T)</code>	<code>fx_forward(pair, t_event, T)</code>	

Surface call	Dispatches to	Notes
cross-ccy receive amt currency=C fx=PAIR	in_currency(...) / multiply by fx_spot(PAIR, t)	convert the tagged amount into the product currency before discounting

6.3 Rate built-ins → the rate door

All interest-rate built-ins resolve through the single rate door (`bond_price / discount_factor / forward_rate / cash`):

Surface call	Resolves via	Meaning
DF(ccy, t_pay)	discount_factor(ccy, t_event, t_pay) (= bond_price)	per-path discount factor to t_pay
LIBOR(ccy, t_s, t_e[, tenor])	forward_rate(ccy, t_event, t_s, t_e)	simply-compounded forward rate for [t_s, t_e]
RFR(ccy, t_s, t_e)	compounded forward_rate over the sub-period	risk-free-rate compounding
OIS(index, t_s, t_e)	compounded-in-arrears forward_rate fold	overnight-index swap rate
CMS(ccy, t_s, n, freq[, tenor])	a swap-rate fold built from forward_rate/bond_price over the n periods	constant-maturity swap rate
inflation (YOY_INFLATION, ZC_INFLATION_SWAP_RATE)	the inflation curve door (analogous forward construction)	

For deterministic rates these reduce to the initial curve; for stochastic rates they read the simulated rate factor. The port must build the same quantities from the **same** rate factor so that `bond_price/forward_rate` match.

6.4 State-mutation primitives

These read/write the per-path state map and return a `Vec`. They **self-seed**: on first call for a label they create `state[label]` from `init` (default 0), then update it.

Primitive	Effect	Returns
SNAPSHOT(label, value)	state[label] ← value (stored for the rest of the path)	the stored value
RUNNING_SUM(label, x, init=0)	state[label] ← state[label] + x	the running sum
RUNNING_MAX/MIN(label, x, init=±∞)	element-wise max/min into state[label]	the running extremum
RUNNING_MEAN(label, x)	online mean (sum + count labels)	the running mean
SUP/INF(label, asset)	running max/min of an asset level (lookback)	the extremum so far
*_IF(label, cond, x, ...)	accumulate only on paths where cond is true	the gated accumulator
BARRIER_SURVIVAL(label, value, barrier, dir, ...)	update a 0/1 survival latch (discrete or continuous/ Brownian-bridge monitoring)	the survival indicator

The string `label` is matched **exactly** (case-sensitive). A single call inside an expanded `for/schedule` therefore accumulates across all iterations, because each iteration’s event reuses the same label.

6.5 Credit / survival built-ins → credit methods

Surface call	Dispatches to
<code>IS_ALIVE(issuer)</code>	<code>alive(issuer, t_event)</code>
<code>SURVIVAL(issuer, T)</code>	<code>forward_survival(issuer, t_event, T)</code>
<code>RECOVERY(issuer)</code>	<code>recovery_rate(issuer)</code>
default-loss / basket-loss families	combinations of <code>default_time</code> , <code>recovery_rate</code> , <code>survival</code>

6.6 Aggregators and shaping

Pure element-wise and reduction functions (`max`, `min`, `abs`, `sqrt`, `exp`, `log`, `clamp`, `cap_at`, `floor_at`, `call_payoff`, `put_payoff`, the rainbow `BEST_OF`/`WORST_OF`/`*_EXCLUDING` families, `AVG`/`SUM` over lists, `sigmoid`, `softmin/softmax`, `INDIC`, ...) need **no** market access: they are closed-form on their `Vec` arguments. Their exact formulas and aliases are in the Payoff Reference, Part 3; an implementation reproduces those formulas directly. They are listed here only to delimit the boundary: everything in §6.2–§6.5 touches the market or state; everything in §6.6 does not.

7. The execution model

The resolved product is run in two stages. This document specifies the **forward state pre-pass** and the **callable contract**; the backward Longstaff-Schwartz sweep that consumes the callables is the Engine Specification’s subject.

7.1 Total ordering: (date, seq)

Every event has a `date` (year-fraction) and an integer `seq`. The event list is sorted by the tuple `(date, seq)`, which gives a **total order** even when several events share a base date:

- Within an expanded block, statements keep declared order via increasing `seq`.
- When an outer decision and a flow share a base date, the outer event takes `seq = 0` and the flow events follow (`seq = 1, 2, ...`).
- This replaces any need to perturb dates by ϵ ; co-dated events stay distinct and ordered without moving them on the time grid. (The only ϵ nudge in the whole pipeline is the cum-div snapshot at `t_ex - 10-9` of §4.5, which is a genuine pre-drop instant, not an ordering hack.)

7.2 The forward state pre-pass

Before pricing, walk the events in `(date, seq)` order **forward**, maintaining the per-path state map:

1. Seed each declared state `label` at `t0` by evaluating its `init` expression (§5) and storing the resulting `Vec`.
2. For each event in order, obtain the per-date snapshot `snap = md.snapshot(date)` and build `env = make_env(snap, state, date)` (§6.1).

3. **Fixing event:** evaluate each `name = expr` assignment in the `do` string, left to right, writing `state[name]` ← the result. Self-seeding built-ins (`RUNNING_*`, `SNAPSHOT`, ...) create their labels here on first use.
4. **Payment / Decision events** do **not** mutate state in this pass; their amounts/intrinsics are captured as **callables** (§7.3) to be invoked later by the engine, after the state trajectory is known.

After this pass, every state label has a value at every event date (the market object can resample it off-grid per §2.2), and every Payment/Decision carries a callable that closes over the *resolved expression* and the state-access pattern.

7.3 The payoff-callable contract

Each Payment exposes `amount(snap, state) → Vec` and each Decision exposes `immediate(snap, state) → Vec`, `alive(snap, state) → Vec`, `forced_stop(snap, state) → Vec`. Each:

- rebuilds `env = make_env(snap, state, t_event)` (§6.1),
- evaluates its resolved expression (§5) against that `env`,
- returns the per-path `Vec`.

Because the expression is already lowered and the market coupling is entirely in `make_env`, the callable is a pure function of `(snap, state)`. The engine invokes it during the backward sweep with the snapshot and the already-built state for that date. A Payment also carries the boolean `discounted` flag and any cross-currency tag; the engine applies discounting and FX (the callable returns the **native** per-path amount, already FX-converted to the product currency if the event was tagged).

Set-after-pay ordering. Within one base date, a `set` placed after a `receive` gets a higher `seq`, so in the forward pass the payment's `seq` is processed first and reads the *pre-update* state — letting a product pay this period's coupon and then roll the state forward in the same block. (Equivalently, a Fixing carries a `do_when ∈ {before, after}` knob, default `before`, expressing the same ordering inline.)

7.4 What the engine receives

The engine receives the `Product`: the ordered event list with, per event, the date, `seq`, kind, the callable(s) above, the `discounted` flag, and — on decisions — the polarity (`max/min`), the `forced_stop/alive` callables, the regressor list, and the physical-settlement flags. From there the Engine Specification takes over (backward recursion, regression, profile reduction).

8. Reproduction checklist

To reimplement the front end from scratch in any language and reproduce the reference per-path amounts:

1. **Parser** for the surface grammar (Payoff-Reference Part 6): comments, date atoms, the expression EBNF.
2. **Lowering (§4)**, in order: constant/let folding; **static-vector element folding + bare-vector list expansion + the SIZE/LEN/NROWS/NCOLS fold; compile-time mutable tracking** (scalar/vector, with the compile-time-only RHS guard and the no-

schedule-block / no-runtime-if control-flow guard); func inlining; comprehension/range unrolling; schedule and schedule_div expansion (including the cum-div snapshot at $t_{ex} - 10^{-9}$ and the iterator loop-var binding); dividend-macro substitution; ratio/perf anchor rewrite; instrument self-expansion; cross-currency tagging; (date, seq) assignment.

3. **Evaluator (§5)** with the exact element-wise / mask / ternary semantics and the positional-vs-keyword call rule.
4. **make_env and the dispatch tables (§6)** — the binding of bare underlyings to `snap[name]`, state labels to the state map, and every market/rate/credit built-in to its `SimulatedMarketData` method.
5. **State pre-pass (§7.2)** in (date, seq) order, with self-seeding accumulators and the set-after-pay ordering rule.
6. **The callable contract (§7.3)** so the resulting `Product` plugs into the Engine Specification's pricing core.

A `SimulatedMarketData` implementation satisfying §2 (the same simulated paths, the same resampling rules, the same rate door) plus the six items above is sufficient to produce a `Product` whose callables return the reference engine's per-path amounts.

8.1 What lives in the three companion documents

Concern	Document
Surface syntax, operator/function catalogue, EBNF, date tables	DSL Payoff Reference
Front end: lowering, evaluation, market dispatch, state model (this)	.amc Execution Specification
Backward LSM recursion, regression, two-pass, PV-profile / PFE, cross-measure	AMC Engine Specification
The query surface itself (<code>SimulatedMarketData</code> methods, invariants)	Engine Spec, Appendix B (and §2 here)

Together they specify the entire path from `.amc` source text to the PV/PFE grid, independent of implementation language.

9. Counterparty exposure (CCR) and the closed-form route

The forward mark-to-market at a future date `a` is the conditional expectation of the remaining cashflows given the path state at `a`; for CCR we want its distribution across **P** paths under a **Q** valuation. `bank_pv_paths` produces it:

```
bank_pv_paths(deal, smd_q, profile_dates, *, smd_p=None, regressor_names=None,
              degree=2, basis='polynomial', control_variate_fn=None,
              force_regression=False, ...) -> ndarray # (n_paths, n_profile)
# bank_direction = -counterparty_direction
```

9.1 Routing

```
if (analytical pricer resolves for the deal) and not force_regression:
    return bank_direction * pricer.evaluate(smd_p, profile_dates) # closed form
on P
else:
    cv = control_variate_fn or (auto from product._cdo_control_variates)
```

```
fits = regress_pv_profile(... control_variate_q = cv(smd_q) ...) # fit on Q
return apply_regression(fits, ... control_variate_p = cv(smd_p) ...) # apply on P
```

9.2 Control variate and the auto-registry

With an F_a -measurable closed form `cv`, $MtM(a) = cv(a) + E_a[\text{realised} - cv(a)]$ is unbiased for any `cv` (subtracted from the regression target before the fit, added back after; threaded as `control_variate_q / control_variate_p`). When a script declares a CDO block the resolver emits `cdo_instruments`, `compile_script` builds a `tranche_pv_paths` closure per instrument and attaches `product._cdo_control_variates`, and `bank_pv_paths` auto-assembles `control_variate_fn` when none is passed — so a mixed netting set gets the CDO control variate automatically while other deals regress normally.

9.3 The `pricing = closedform` bypass

`AnalyticalPricer(name, pv_fn)` with `resolve_analytical_pricer_for_deal(deal)`. Setting `pricing = closedform` wraps the per-instrument closed forms as an analytic pricer (`product._cdo_analytical_pricer`); `bank_pv_paths` then returns `bank_direction ×` the closed form on `P` with **no regression** (verified exact, `max|diff| = 0`, and decaying to runoff). `pricing = analytic` is a separate advisory value that stays on the regression route. The path-gate guard (Appendices, Appendix D) refuses `closedform` when the CDO value is gated.

9.4 What agrees, what doesn't

The two routes agree in **mean** (with the corrected EL kernel, $\sim 0.002/\text{unit}$ at independence). For the **distribution** (EPE/PFE) they agree only with the control variate: a bare polynomial regression compresses the tails of discrete tranche-loss jumps, so its EPE/PFE sit below the closed form; the control variate restores the dispersion (EPE gap-to-substitution cut $\sim 2.6\times$). Any residual PFE difference is the simulation model's dependence (e.g. frailty) differing from the copula at the quoted `rho` — real, not numerical (persists out-of-sample). **Route choice:** un-gated tranche $\rightarrow$ `closedform` (exact, no fit); gated / knock-out $\rightarrow$ control-variate regression.